Expositions: readable mathematics with Lean receipts

1. The problem

The mechanized results (`lean/Adic/*.lean`, seven-plus theorems and growing) are truth without readability; the letters are readable but personal correspondence — deliberately not the place for systematic exposition (Mathijs, 2026-08-09: don't force this into letter form). Composix solved the analogous problem for CLI documentation with *tours*: prose whose every command really ran and whose outputs are asserted by the build. Wanted: the mathematical analogue — human mathematics, machine receipts.

2. Prior work

- **Composix tours** (`docs/tour/`): auto-generated by a test harness; "every command below really ran"; outputs asserted; drift structurally impossible. The mechanism to copy is not the generator but the *assertion*: prose is pinned to checked artifacts.
- **Verso** (Lean's documentation framework, used for the official Lean books): prose documents in which embedded Lean is really elaborated — drift-proof by construction. The right spirit; the cost is authoring in Lean and leaving our Typst toolchain.
- **Literate proofs / blueprint** (Massot's leanblueprint for big formalization projects): human TeX statements linked to Lean declarations with dependency graphs — the closest existing genre; its linkage is by name only, not statement-pinned.

3. Recommendation

`expo/` at the repo root: numbered expository chapters in **Typst** (cited as "Expo 1"), impersonal register, cannibalizable by the paper exactly like letters. The load-bearing novelty is the **receipt box**: wherever a theorem is presented, the document shows human notation (prose + Typst math) alongside a machine-injected box containing the Lean name, the pretty-printed formal statement, and its `#print axioms` set.

Mechanism (the tour move, translated):

1. A small Lean metaprogram (`lake exe receipts`, or an `@[expo]` attribute + collector) dumps `receipts.json`: for each exported declaration, its name, pretty-printed statement, and axiom set.
2. Typst reads the JSON (`json()`) and renders receipt boxes; a receipt referenced in prose that is absent from the JSON fails the compile — citing a nonexistent theorem breaks the build.
3. **Statement pinning**: the Typst source records a hash of each cited statement. If the Lean statement changes, the build fails until the author re-reads and re-pins — the exact analogue of the tour's asserted outputs. Prose can still lie about what a statement *means*, but never about what it *says* or whether it is proved, and every statement change forces a human re-read.
4. Claims without a receipt must carry an explicit badge (`unformalized` / `desk-proved`) — this absorbs the deferred paper's claims-ledger: an exposition IS the ledger, rendered.

Gate: `expo/*.typ` compile (with receipt resolution) joins the gate next to letters.

4. Taste calls (all decided by Mathijs, 2026-08-09)

1. **Genre name.** DECIDED: expositions (`expo/`, "Expo 1"). Additionally: expositions largely *replace letters* as the precursor vehicle — existing Letters 1–2 get reformulated as expos; the letters directory remains as immutable correspondence archive (AIP-3 disposition note). An expo need NOT have Lean receipts — but unreceipted mathematics must be clearly badged as such.
2. **Pin hardness.** DECIDED: hash-pin. Mathijs's framing: an expo *claims*, the Lean code *backs*, and the claim⟶backing path is CI-checkable — assertable and strikeable by the build.
3. **Authoring surface.** DECIDED: Typst first, with nice reusable functions and a distinct visual style for "Lean-backed" vs "plain mathematics" (`expo/lib.typ`). Verso stays the fallback if the harness grows painful.
4. **First exposition + authorship.** DECIDED: harness track first, Expo 1 ("The dyadic machine") after. Claude writes expos with its own hand; Mathijs corrects rather than pre-approves ("ik corrigeer ipv accordeer").
5. **Definitions are first-class** (AMENDED, Mathijs 2026-08-10): expos should present the *objects* — definitions, notation, the model's ingredients — as ordinary mathematical prose, not only the theorems. Definitions carry no badge and need no Lean backing: they claim nothing, they set the stage; every *claim* about them keeps its badge discipline. `expo/lib.typ` provides `#defn` for a consistent light visual (no box — boxes stay reserved for auditable claims). Applies to all new expos; existing expos get a definitions retrofit pass.